22. State Abstraction

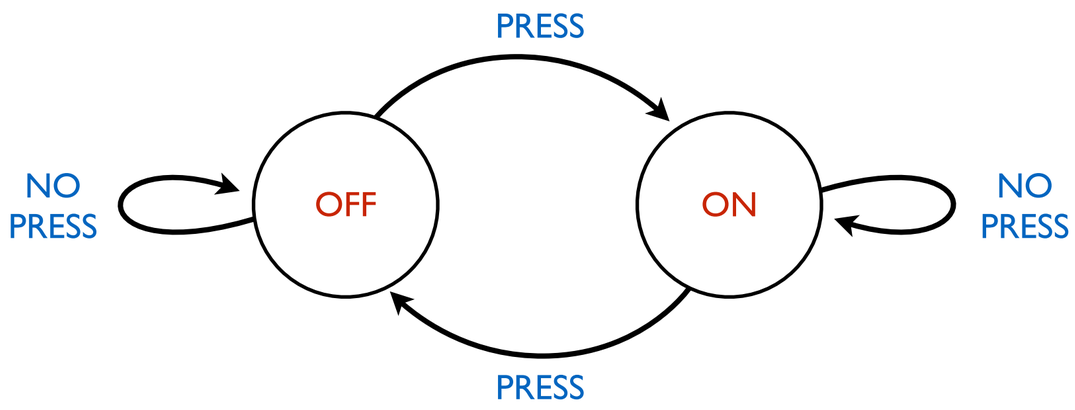
The Concept of State

The concept of a state is at once one of the most profound and one of the most trivial ideas we will explore in this class. On the trivial end, identifying a machine's states is little more than some tedious book-keeping. (And that'll be the focus of this lesson.)

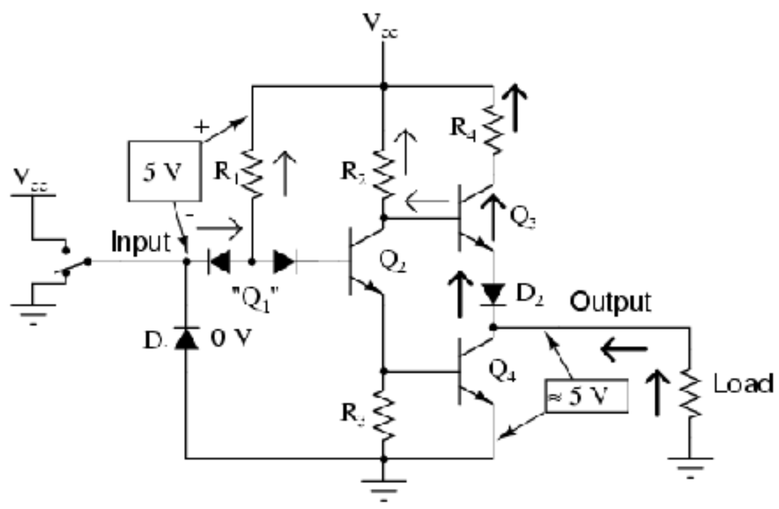
On the profound end, once defined, the concept of a machine state provides us with fundamental insights about the relationship between low-level physics and high-level cognition. The philosopher Hillary Putnam even thought that the concept of machine states solved (or dissolved!) the mind-body problem. You'll have to judge for yourself.

A **state** is the overall configuration of a given system. Given an input, a system's state determines its current output and what state it will be in next. **State abstraction** leaves aside particular details of circuitry, and describes system purely in terms of inputs, outputs, and state transitions.

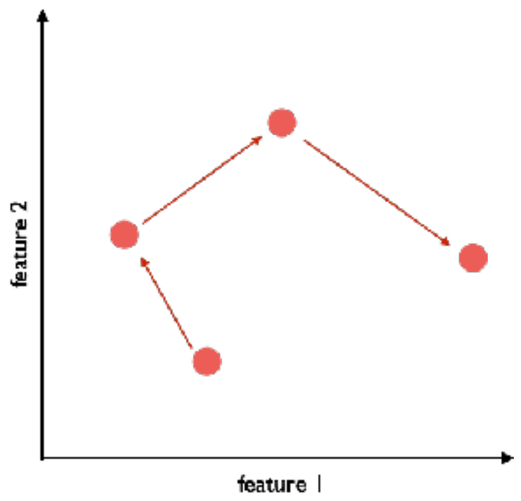
For example, here is a diagram which gives the state abstraction of the on-off switch. It describes the machine entirely in terms of its overall dispositions to produce inputs in response to outputs.



ON/OFF Switch



Whereas a machine as a complicated internal configuration at any given point…



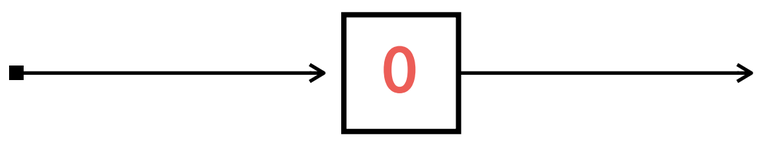
We can imagine a **state space** of all possible internal configurations. Each state is a point in this space.

States of an SC

In the case of sequential circuits, a state is the total "input-output disposition" of the machine at a given moment. It includes how the machine will respond to all possible inputs at a time. It also includes what states the current state will lead to. It **doesn't** include the actual circuity of the machine.

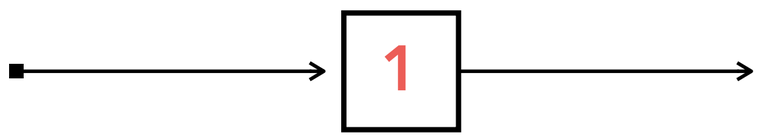
Note: there is an ambiguity about whether two different memory settings, with same next state, and same I/O should be counted as "same state"

The total input-output disposition of a machine is determined by the internal circuitry of the machine and the configuration of its memory registers at a time. All machines start in the "initial state," where every memory register is set to 0. This determines one set of responses to possible inputs. Other settings of the memory registers result in different responses to possible inputs, hence different states.



State 0

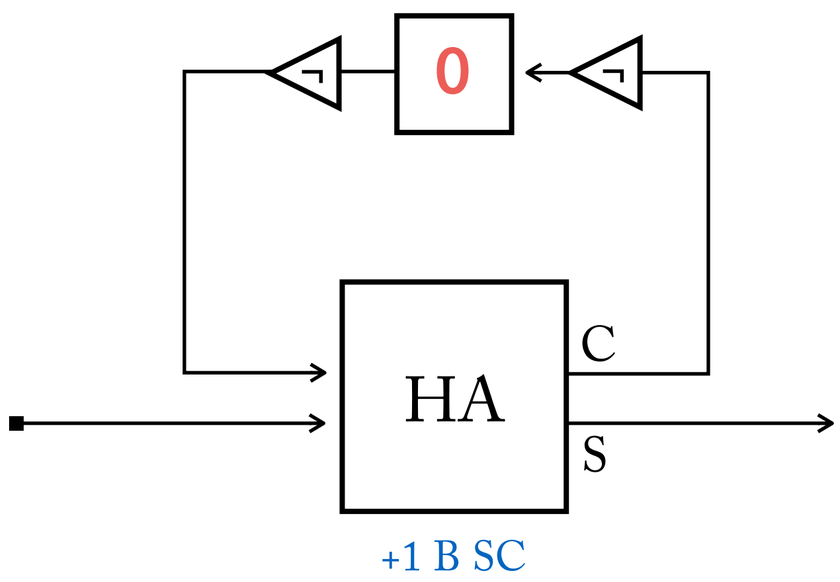
IN	OUT
1	0
0	0



State 1

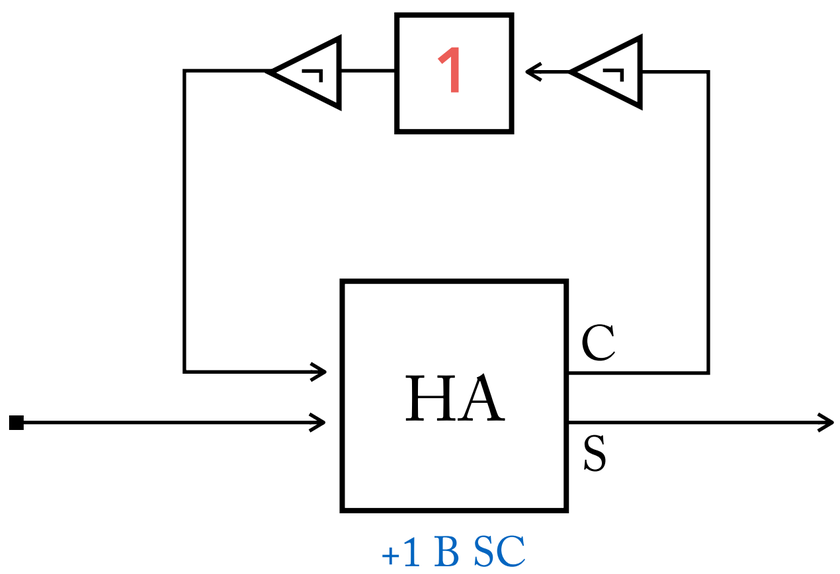
IN	OUT
1	1
0	1

Here's a more interesting example:



State 0

IN	OUT
1	0
0	1



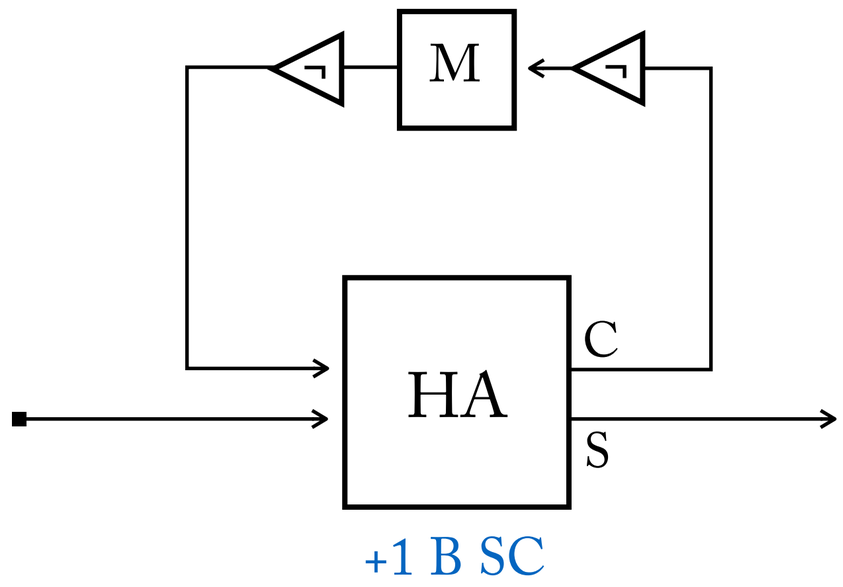
State 1

IN	OUT
1	1
0	0

But there is more to a state that just input and output. There is also its **relation to other states**…

State Tables

Step 0
Design a sequential circuit which computes a specific function.



Step 1
Make a table of all possible inputs from memory and inputs from the user.

If there are more memory registers, or more inputs from the user, there will be more rows in the table.

M→	IN	OUT	→M
0	0	1	1
0	1	0	0
1	0	0	1
1	1	1	1

Step 2
Fill out the table with all of the outputs to the user and all of the outputs to memory.

Step 3
Add a column at the left for "**current state**" and a column on the right for "**next state**".

Think of the input from M→ as describing the current *total memory configuration* of the machine.

CURRENT STATE	M→	IN	OUT	→M	NEXT STATE
S_0	0	0	1	1	A
S_0	0	1	0	0	S_0
A	1	0	0	1	A
A	1	1	1	1	A

Step 4
For each distinct possible configuration of memory, enter a distinct state name in the "**current state**" column. For the configuration where all memories are set to 0, that state should be called S_0. After that you can call the states anything you like.

Step 5
Using the state names chosen in Step 4, fill out the "**next state**" column. The value of →M determines the next value of M→, which determines the identity of the next state.

Step 6
Merge the cells in the "**current state**" column that correspond to the same state.

CURRENT STATE	M→	IN	OUT	→M	NEXT STATE
S_0	0	0	1	1	A
	0	1	0	0	S_0
A	1	0	0	1	A
	1	1	1	1	A

Step 7
Erase the memory columns.

CURRENT STATE	IN	OUT	NEXT STATE
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

… and you've got yourself a **state table**!

Step 8 (Optional)
Eliminate unused states: eliminate any state that either:
(1) cannot be reached by finite steps, beginning with S_0;
(2) will not be reached by finite steps, beginning with S_0, given the representational code (binary, tally) intended for this machine.

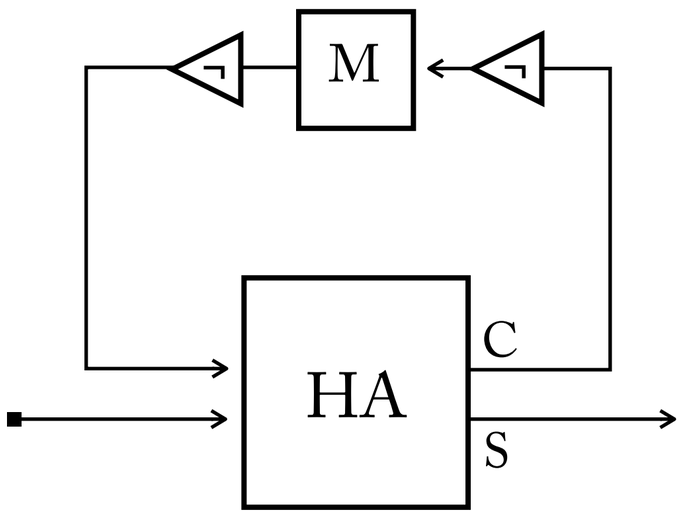
Note: this step is not *strictly* required, but it is best practice.

For purposes of illustration, here is a table for a machine with two memory registers, before the memory columns have been erased.

STATE	M1	M2	IN	OUT	NEXT
S_0	0	0	1	1	S_0
			0	1	A
A	0	1	1	0	S_0
			0	1	C
B	1	0	1	0	B
			0	0	C
B	1	1	1	1	C
			0	1	S_0

The grey row is eliminated by Step 8(1).

From State Tables to State Diagrams



+1 B SC

+1 B SC determines the following state table.

STATE	IN	OUT	NEXT
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

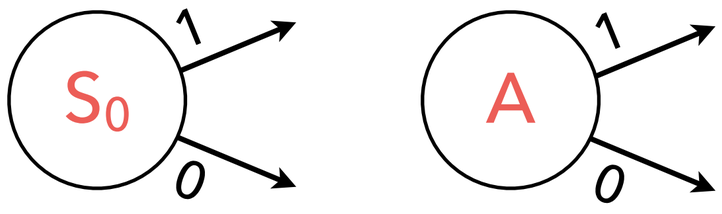
+1 B State Table

To construct a state diagram from a state table…

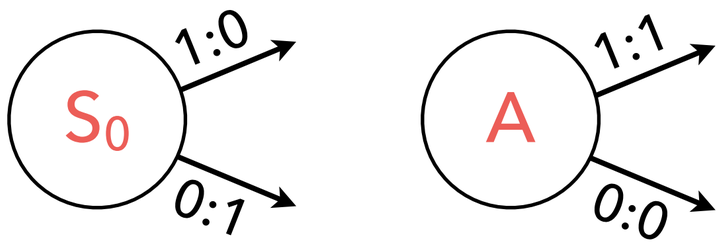
Step 1
For each state of the machine draw a circle, and label it with the name of the state.



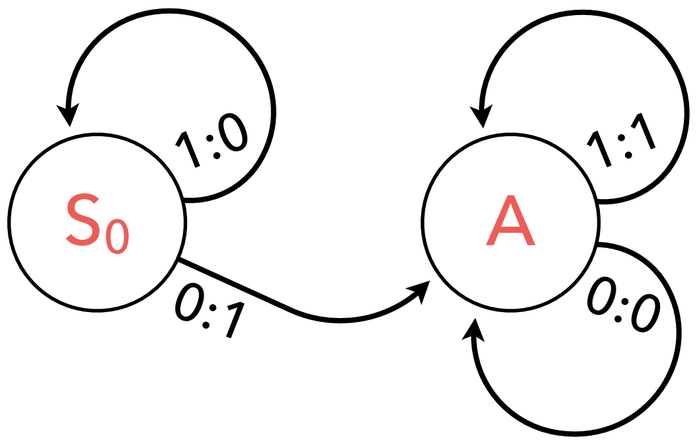
Step 2
For each circle draw up to two arrows exiting it, one labeled "1", the other "0" corresponding to the two possible inputs for that state.



Step 3
For each arrow, add the notation "input : output", corresponding to the input and output associated with the given state transition.



Step 4
For those inputs which lead to a next state, connect the arrow to that state.

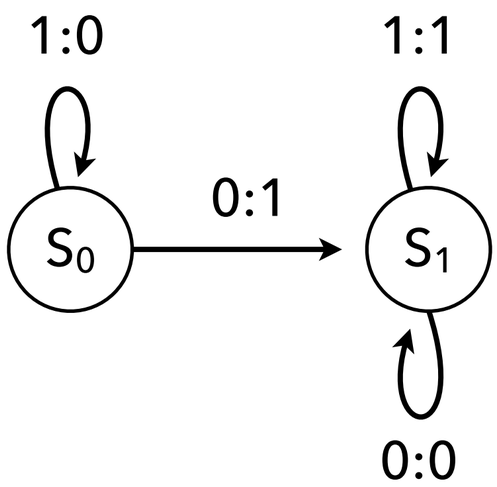


Note: you'll often find diagrams with some of the I/O arrows omitted because they don't arise in computation for a given problem.

STATE	IN	OUT	NEXT
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

+1 B State Table

equivalent descriptions



+1 B State Diagram